

Project Proposal: SecureAssist – Gen-AI Powered Security & Help Assistant 1) Goal of Your Project

The primary goal of the **SecureAssist** project is to deliver a reliable, authoritative, and instantaneous source of information regarding the organization's security policies and general cyber hygiene best practices.

- **Problem Solved:** Traditional support methods (email, phone calls) are often slow and costly for high-volume, repetitive security questions. Generic chatbots frequently "hallucinate" or provide unsafe, incorrect external information.
- **Key Objective:** To implement a **Retrieval-Augmented Generation (RAG)** system, powered by the **Gemini API** and **File Search** tool. This system strictly grounds all responses in the company's official, updated security manual (security_docs.txt), ensuring compliance and factual accuracy.
- **Measure of Success:** The assistant must successfully retrieve and generate accurate answers for queries ranging from specific internal policies (e.g., 2FA setup) to complex security concepts (e.g., SQL Injection), and gracefully decline to answer topics not covered in the internal documents.

2) Design Patterns

The project utilizes two primary, industry-standard design patterns to ensure a secure, robust, and reliable AI application.

A. Retrieval-Augmented Generation (RAG) Pattern

- **Description:** The RAG pattern is the core intelligence architecture of the assistant. It mitigates the risk of the Large Language Model (LLM) producing factually incorrect or outdated information (hallucinations).
- **Mechanism:** When a user submits a query, the system first retrieves the most semantically relevant information from the indexed knowledge base (security_docs.txt). This retrieved context is then passed directly to the Gemini model along with the user's question, instructing the model to generate its answer *only* using that provided textual evidence.

B. Thin Backend / Service-Oriented Architecture (SOA)

- **Description:** The Flask application (app.py) serves as a minimalist, **thin backend** that acts primarily as a secure proxy between the user interface and the Gemini API.
- **Implementation:** The core intelligence (embedding, search, generation) is offloaded to the highly efficient Google Gemini service. This separation ensures

the backend remains lightweight, fast, and that sensitive logic (API keys, document indexing) remains securely on the server side.

3) Technology Used

The project is built using a modern, efficient technology stack:

- **Generative Model: Gemini** ◦ *Role:* The Large Language Model (LLM) responsible for natural language understanding and transforming retrieved text into a coherent, humanreadable answer.
- **Knowledge Base (RAG): Gemini File Search Tool** ◦ *Role:* Handles the entire RAG pipeline: it chunks, embeds, indexes, and retrieves the most relevant security policy text from the uploaded security_docs.txt file.
- **Backend Framework: Python / Flask** ◦ *Role:* A lightweight web server that handles user HTTP requests, manages session state, and securely communicates with the external Gemini API.
- **Data Persistence: SQLite** ◦ *Role:* A simple, file-based database used to store the **Interaction Log** for compliance and auditing purposes.
- **Frontend: HTML, CSS, JavaScript** ◦ *Role:* Provides a responsive, familiar, and functional chat interface for the end-user.

4) Database Schema

The project uses a simple relational database (SQLite) to maintain a complete, auditable log of every interaction between the user and the assistant. **Table:**

InteractionLog

- **Column:** id ◦ **Data Type:** INTEGER (Primary Key) ◦ **Description:** Unique identifier for each logged interaction.
- **Column:** timestamp
 - **Data Type:** TEXT
 - **Description:** Date and time the interaction occurred (recorded as YYYYMM-DD HH:MM:SS).
- **Column:** user_query ◦ **Data Type:** TEXT ◦ **Description:** The exact question or prompt submitted by the end-user.
- **Column:** ai_response ◦ **Data Type:** TEXT ◦ **Description:** The complete answer generated by the Gemini model, based on the RAG process.

- **Column:** store_name ○ **Data Type:** TEXT ○ **Description:** The unique ID of the specific RAG knowledge base used, ensuring traceability.

Source Code:

Github Link :

<https://github.com/appu415/Secure-Assist>